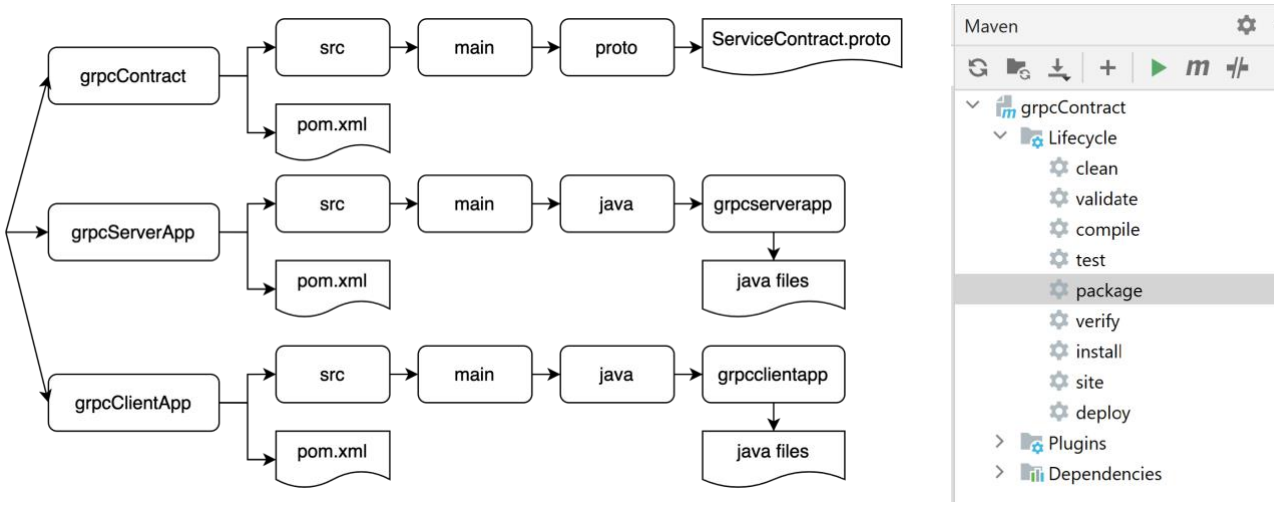
Laboratório 2

Objetivos:

- Desenvolver serviços distribuídos com tecnologia *Google Remote Procedure Call* (gRPC)
- Distinguir entre chamadas unárias e chamadas com *stream* de servidor

Parte 1

1) Considere o exemplo dos três projetos gRPC apresentados e demonstrados na aula (anexo `grpc-Slides-Example.zip`), com a estrutura apresentada na Figura 1a). Os projetos `grpcServerApp`, `grpcClientApp` e `grpcContract` têm a estrutura de diretorias Maven com o respetivo pom.xml. O projeto `grpcContract` tem o contrato definido no ficheiro `NumbersServiceContract.proto` que, por convenção do compilador de `protobuf`, está localizado na diretoria `\src\main\proto`. Veja abaixo (secção anexo) como abrir os projetos Maven no IntelliJ.



a) b)

Fig. 1: a) Estrutura de diretorias e ficheiros dos projetos fornecidos; b) Janela Maven no IntelliJ

No projeto `grpcContract`, usando o menu Maven no ambiente IntelliJ, gere o JAR com o contrato (ação ***package***) e instale-o no repositório local do Maven (ação ***install***).

2) Compile o código fonte do servidor e do cliente, e gere os respetivos JAR através da opção ***package*** do Maven. Verifique que não há erros de compilação e teste o servidor e o cliente experimentando as diferentes operações disponíveis no contrato. Pode executar ambas as aplicações diretamente no IntelliJ ou através da linha de comandos, usando os JAR obtidos anteriormente.

3) Se o servidor não implementar a operação `isAlive` do contracto a operação estará mesmo assim disponível no *stub* do cliente?

4) Acrescente uma nova operação para determinar que números primos existem num determinado intervalo:

   a) Acrescente ao contrato a seguinte operação, com *stream* de servidor, cujo objetivo é obter os números primos que existem no intervalo [start; end]:

```
rpc findPrimes(IntervalNumbers) returns (stream IntNumber)
message IntervalNumbers { int32 start = 1; int32 end = 2; }
```

   b) Implemente a nova operação do contrato no servidor.

   c) Desenvolva uma nova aplicação cliente que, usando um *stub* não bloqueante, realiza 5 chamadas à operação `findPrimes` para calcular os números primos entre 1 e 500, em intervalos de 100 números ([1;100], [101;200], …). Note que, como as chamadas são realizadas com um *stub* não bloqueante, a chamada seguinte é feita sem que a anterior tenha já sido concluída (ou seja, sem esperar que sejam recebidos todos os números primos da chamada anterior). Também por este motivo, as respostas chegarão em paralelo ao cliente, apesar de não ser necessário gerir explicitamente *Threads* nem no lado do cliente nem no lado do servidor.

   d) Após testar na sua máquina local, crie o artefacto do projeto servidor (opção *package* no menu Maven), copie o artefacto (JAR com todas as dependências) para uma VM no seu projeto GCP e teste as aplicações cliente e servidor neste cenário de *deployment*.

Parte 2

1) Considere um sistema para troca de mensagens (*Forum*) cujo envio e receção de mensagens está organizado em tópicos. O contrato do serviço a disponibilizar pela aplicação servidora é apresentado em seguida:

```
syntax = "proto3";
import "google/protobuf/empty.proto";
option java_multiple_files = true;
option java_package = "forum";
package forum;

service Forum {
   // subscribe a topic
   rpc topicSubscribe(SubscribeUnSubscribe) returns (stream ForumMessage);
   // unsubscribe a topic
   rpc topicUnSubscribe(SubscribeUnSubscribe) returns (google.protobuf.Empty);
   // get all topics in server
   rpc getAllTopics(google.protobuf.Empty) returns (ExistingTopics);
   // send a message to a topic
   rpc publishMessage(ForumMessage) returns (google.protobuf.Empty);
}

message SubscribeUnSubscribe {
   string usrName = 1;
   string topicName = 2;
}
message ExistingTopics {
   // list of topic names
   repeated string topicName = 1;
}
message ForumMessage {
   string fromUser = 1;
   string topicName = 2;
   string txtMsg = 3;
}
```

O contrato representa as seguintes funcionalidades:

- `topicSubscribe`: Operação que recebe o identificador único de um utilizador e o nome do tópico que ele pretende subscrever (ex: desporto, economia, política). A partir desse momento, o utilizador recebe as mensagens enviadas para esse tópico. É criado um tópico novo se ainda não existir nenhum com o nome indicado.
- `topicUnSubscribe`: Operação que recebe o identificador do utilizador e o nome do tópico que pretende cancelar a subscrição.
- `publishMessage`: Operação que recebe o nome do utilizador, o nome do tópico e a mensagem de texto a ser publicada no tópico. Só é permitido publicar mensagens em tópico já subscritos. Se o utilizador não tiver subscrito o tópico deve ser enviado informação de erro para o cliente.
- `getAllTopics`: Operação que retorna uma lista com os nomes de todos os tópicos de discussão que existem no servidor.

a) Implemente a aplicação servidora que disponibiliza o serviço *Forum*. Sugere-se que a gestão do estado do servidor relativo a tópicos e utilizadores conectados seja suportada pela seguinte estrutura de dados:

```
// map: { (topic, map: { (username, streamObserver) } ) }
ConcurrentMap<String, ConcurrentMap<String, StreamObserver<ForumMessage>>>
```

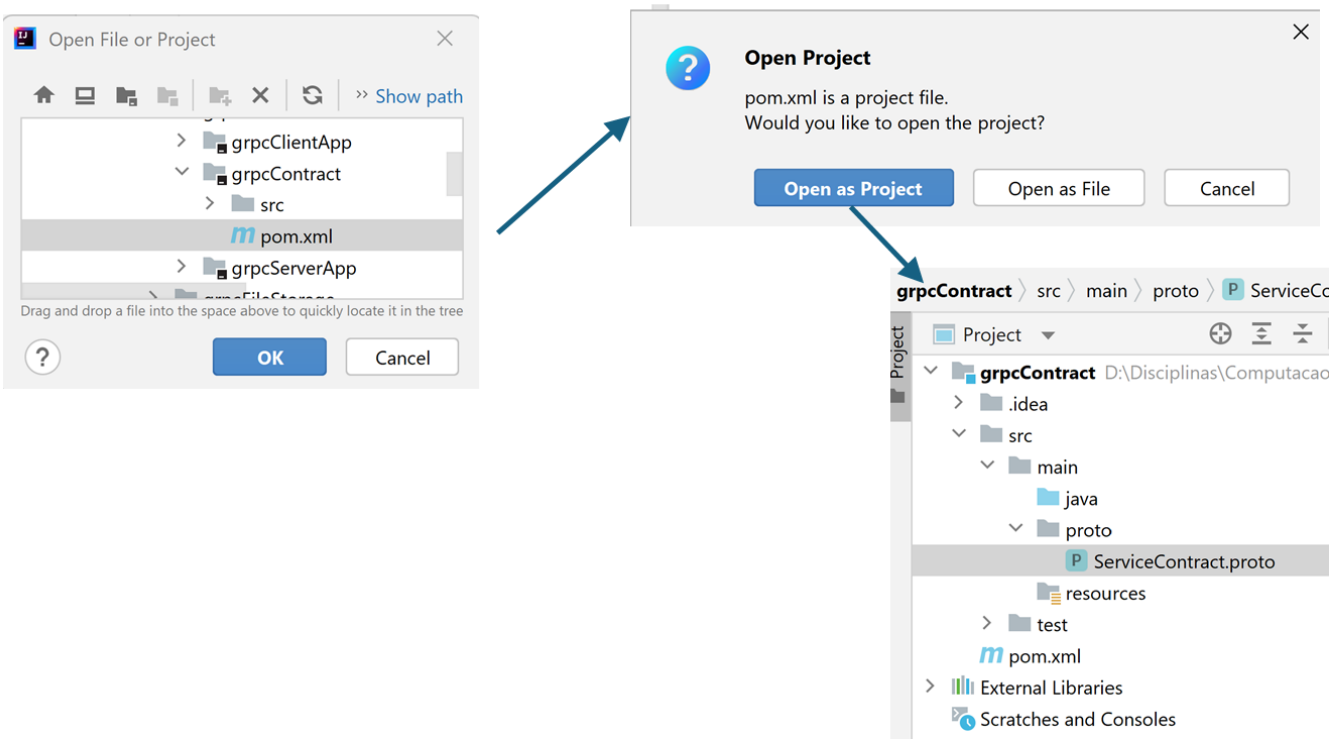
Esta estrutura consiste num *hash map* cuja chave é o nome de um tópico e cujo valor é outro *hash map* cuja chave é o nome dos utilizadores, que subscreveram o tópico, e o valor é o respetivo *stream observer* onde um utilizador receberá as mensagens do tópico que subscreveu.

b) Implemente uma aplicação cliente, em modo consola, que permita aos utilizadores trocarem mensagens em vários tópicos.

c) Teste o sistema executando o servidor numa VM do GCP e clientes no computador pessoal de cada elemento do grupo.

d) *Checkpoint*: Na segunda aula dedicada a este laboratório deve disponibilizar o endereço IP da VM onde está a executar o servidor para que o professor possa usar uma aplicação cliente para aceder ao servidor de cada grupo. Registe o IP público da VM neste documento: https://docs.google.com/spreadsheets/d/1p_6yeY6nskkqqCuTtIOCGiT7p3armoCMSuuWWG4ByPE/edit?usp=sharing

ANEXO

Como abrir projetos no *IntelliJ* a partir da descrição Maven (*pom.xml*).

Abrir o projeto Maven selecionando na estrutura de diretorias o ficheiro *pom.xml*